

unicode print library for python

<https://gemini.google.com/app/e97987153e8dc5ef>

User prompt: unicode print library for python

Response: Several Python libraries specialize in formatting and printing Unicode text, tables, colored output, and terminal interfaces.

Rich

The most comprehensive library for rich Unicode text, emoji, syntax highlighting, progress bars, and formatted tables in the terminal.

- **Installation:** `pip install rich`
- **Key Use Case:** Pretty-printing complex objects, formatted text, and Unicode borders.

```
from rich import print
from rich.panel import Panel

# Prints full Unicode range including emojis, styling, and characters
print("[bold magenta]Unicode Test:[/bold magenta] ✨ 🚀 🍷")
print(Panel("✨ Beautiful Unicode Box ✨", title="[bold yellow]Rich Panel[/bold yellow]"))
```

Tabulate

A lightweight, zero-dependency library for printing tabular data with a wide selection of Unicode box-drawing borders.

- **Installation:** `pip install tabulate`
- **Key Use Case:** Displaying clean ASCII or Unicode tables from lists or dicts.

```
from tabulate import tabulate

table_data = [{"Item 1", "✨ 100%"}, {"Item 2", "🚀 250%"}]
headers = ["Name", "Score"]

# "fancy_grid" and "rounded_grid" use Unicode box characters
print(tabulate(table_data, headers=headers, tablefmt="rounded_grid"))
```

Termcolor & Colorama

Minimalist libraries for adding ANSI colors and formatting to Unicode strings in standard terminal outputs.

- **Installation:** `pip install termcolor`
- **Key Use Case:** Simple colored status text without complex layouts.

```
from termcolor import colored

print(colored("✓ Operation Successful! 🚀", "green", attrs=["bold"]))
print(colored("✗ Connection Failed! ⚠️", "red"))
```

Textwrap & Unicodedata (Built-in)

Python's standard library provides modules to handle Unicode character properties, visual alignment, and line wrapping.

```
import unicodedata

# Get details on specific Unicode characters
char = "✨"
print(f"Character: {char}")
```

```
print(f"Name: {unicodedata.name(char)}")
print(f"Category: {unicodedata.category(char)}")
```

User prompt: how about print to pdf using python tools? will it be printed?

Response: Depending on what you mean by "print to PDF," Python supports both sending Python outputs (HTML/Text) to a **PDF file on disk** and sending a document directly to a **physical printer**.

Scenario A: Generate / Render a PDF File

To convert Python output, HTML, or plain text into a .pdf file on disk, these options cover the core use cases:

1. FPDF2 (Simplest for Text & Images)

Lightweight with zero external dependencies. Ideal for generating simple text documents, invoices, or reports directly from Python.

```
from fpdf import FPDF

pdf = FPDF()
pdf.add_page()
pdf.set_font("Helvetica", size=16)
pdf.cell(text="Hello World! 🌟", new_x="LMARGIN", new_y="NEXT")
pdf.output("output.pdf")
```

2. Playwright (Best for Web Pages / HTML / CSS)

Generates high-fidelity PDFs from HTML strings or web pages using a headless browser engine. Handles custom fonts, CSS layout, and Unicode characters cleanly.

```
import asyncio
from playwright.async_api import async_playwright

async def create_pdf():
    async with async_playwright() as p:
        browser = await p.chromium.launch()
        page = await browser.new_page()
        # Render raw HTML string or load a URL
        await page.set_content("<h1>Invoice #101</h1><p>Unicode Test: 🌟 🚀</p>")
        await page.pdf(path="document.pdf", format="A4")
        await browser.close()

asyncio.run(create_pdf())
```

3. ReportLab (Best for Precise Programmatic Control)

Provides low-level pixel alignment on a virtual page canvas. Standard choice for dynamic enterprise documents and complex data reports.

Scenario B: Send a PDF to a Physical Printer

To send an existing PDF file directly to a paper printer, platform-specific system calls are required:

Windows (via win32print)

```
import win32api
import win32print

# Send document directly to the default OS printer
file_path = "output.pdf"
win32api.ShellExecute(0, "print", file_path, None, ".", 0)
```

macOS / Linux (via cups / lp)

```
import subprocess
```

```
# Send PDF directly to the default CUPS printer queue  
subprocess.run(["lp", "output.pdf"])
```

Sources

1. [toprint - PyPI](#)
 2. [The Best Python Libraries for PDF Generation in 2025](#)
-